

mGFS 分布式文件系统

2351044 崔艺洋

2026 年 6 月 10 日

目录

1	项目概述	2
1.1	核心设计目标	2
1.2	技术栈	2
2	系统架构设计	2
2.1	整体架构	2
2.2	RPC 接口 (gfs.thrift)	3
2.3	元数据模型	3
3	关键算法设计	3
3.1	分块分配	3
3.2	租约与主副本选举	4
3.3	写入流水线	4
3.4	读故障转移	4
4	系统实现	5
4.1	目录结构	5
4.2	Master 节点状态	6
4.3	ChunkServer 块存储实现	6
4.4	流水线抽象 (client/stream.py)	6
4.5	配置参数 (gfs-site.xml)	7
5	测试设计与结果	7
5.1	测试环境	7
5.2	测试用例分组	7
5.3	关键测试用例	8
5.4	测试结果	8
5.5	并发测试设计 (T28)	8
6	设计权衡与局限性	9
7	总结	9

1 项目概述

mGFS (Mini Google File System) 是基于 Google GFS 论文 [1] 的简化 Python 实现, 采用 Apache Thrift 作为 RPC 框架, 实现了 GFS 的核心机制: 单主节点元数据管理、64MB 分块存储、三副本写入与基于租约的主副本选举。

1.1 核心设计目标

- 大文件分块: 以 64MB 为单位切分, 支持任意偏移读写与追加。
- 多副本容错: 每块默认 3 副本; 读操作自动故障转移至健康副本。
- 集中式元数据: Master 唯一维护文件-块-位置映射, 全部驻留内存。
- 租约一致性: 60s 基于时间的租约确定主副本, 协调并发写入。

1.2 技术栈

表 1: 技术栈

组件	选型
语言	Python 3
RPC 框架	Apache Thrift (thriftpy2 动态加载 IDL)
并发模型	threading.Lock + 守护线程
配置格式	XML (gfs-site.xml)
测试框架	unittest (30 个用例)

2 系统架构设计

2.1 整体架构

mGFS 沿用 GFS 的三层架构: 客户端 (Client)、主节点 (Master) 和数据节点 (ChunkServer)。所有通信经由 Thrift RPC 完成, Master 与 ChunkServer 分别暴露独立服务接口, 如图 1 所示。

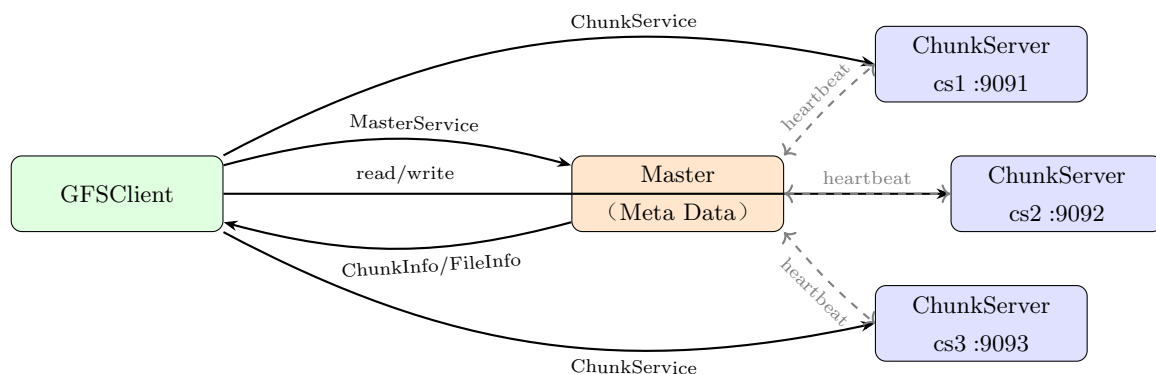


图 1: mGFS 系统架构图

2.2 RPC 接口 (gfs.thrift)

Listing 1: Thrift IDL 核心定义

```
1 struct ChunkInfo {
2     1: string chunk_handle,
3     2: list<string> locations,
4     3: string primary
5 }
6 struct FileInfo {
7     1: string file_name,
8     2: i64 file_size,
9     3: list<string> chunk_handles
10 }
11 service MasterService {
12     bool create_file(1: string filename),
13     bool delete_file(1: string filename),
14     FileInfo get_file_info(1: string filename),
15     ChunkInfo get_chunk_handle(1: string filename, 2: i32 chunk_index),
16     bool heartbeat(1: string server_id, 2: list<string> chunk_handles)
17 }
18 service ChunkService {
19     string read(1: string chunk_handle, 2: i32 offset, 3: i32 length),
20     bool write(1: string chunk_handle, 2: i32 offset, 3: string data),
21     bool append(1: string chunk_handle, 2: string data),
22     bool copy_snapshot(1: string source, 2: string destination)
23 }
```

2.3 元数据模型

Master 在内存中维护两张核心映射表及一张租约表，关系如图 2 所示。

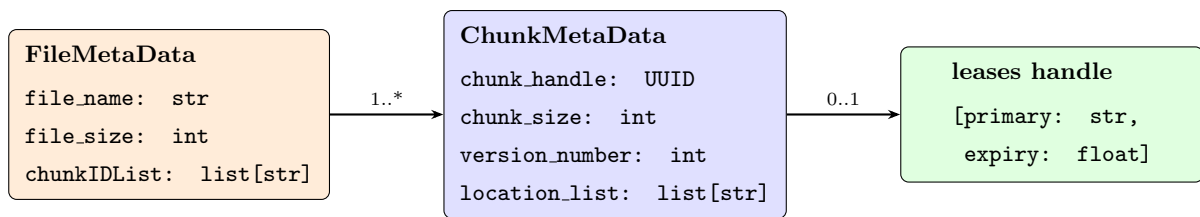


图 2: Master 内存元数据结构关系

3 关键算法设计

3.1 分块分配

当客户端首次访问某块时，Master 在持锁状态下：（1）生成 `uuid4()` 作为全局唯一块句柄；（2）从在线 `ChunkServer` 字典选取前 $\min(N, 3)$ 个节点；（3）写入 `ChunkMetaData`；（4）调用租约模块确定主副本。

3.2 租约与主副本选举

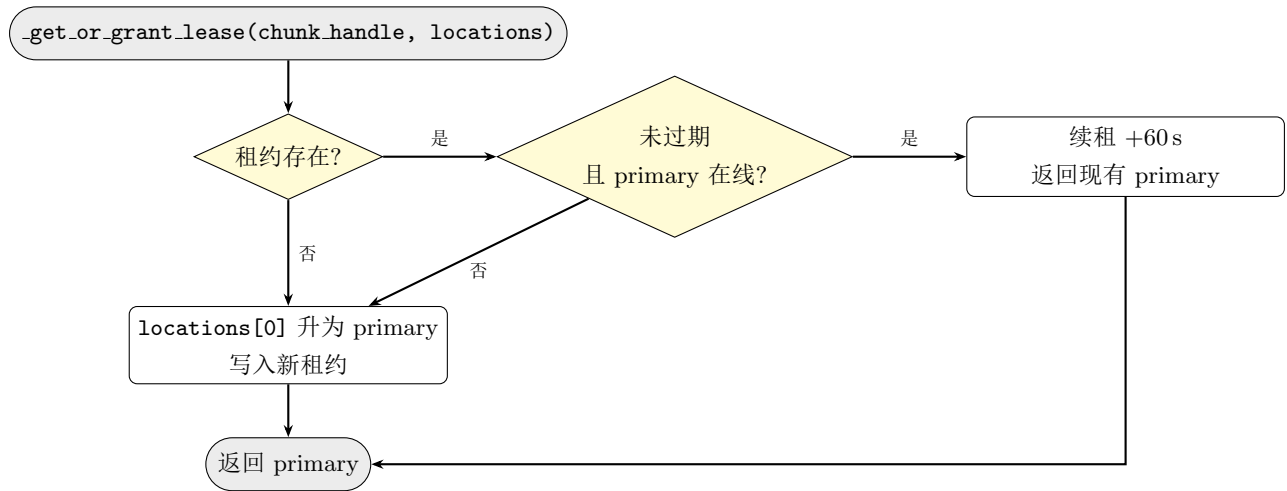


图 3: 租约授予 / 续租流程

3.3 写入流水线

客户端通过 `DataQueue`（数据包缓冲）和 `ACKQueue`（确认追踪）实现写入协议，流程如图 4 所示。

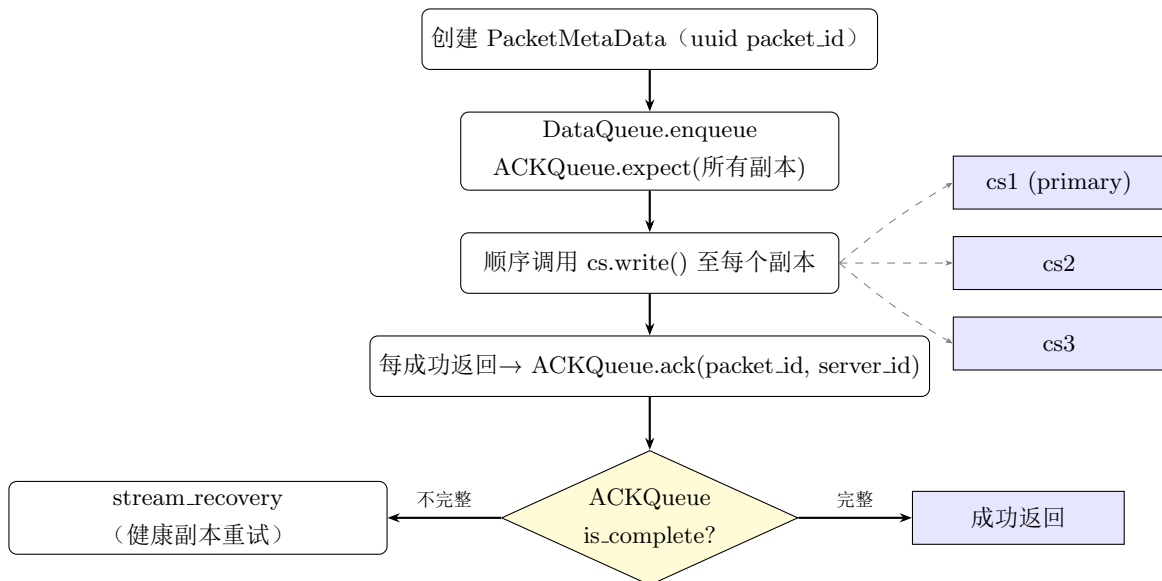


图 4: 写入流水线流程图

3.4 读故障转移

`_read_from_replica` 按副本列表顺序逐一尝试，返回首个成功结果，失败副本静默跳过：

Listing 2: 读故障转移核心逻辑 (client.py)

```
1 def _read_from_replica(self, chunk_info, offset, length):
2     for server_id in chunk_info.locations:
3         try:
```

```

4         host, port = self.chunk_server_ports[server_id]
5         with make_client(ChunkService, host, port) as cs:
6             return cs.read(chunk_info.chunk_handle, offset, length)
7     except Exception:
8         continue
9     return ""

```

4 系统实现

4.1 目录结构

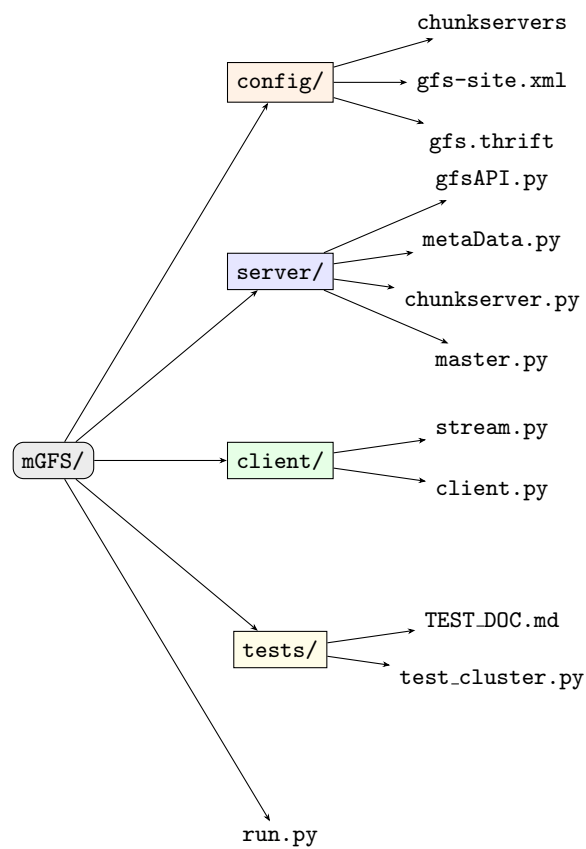


图 5: 项目目录结构

4.2 Master 节点状态

表 2: Master 内部状态表 (server/master.py)

属性	类型	用途
file_meta_data	dict[str, FileMetaData]	filename → 文件元数据
chunk_meta_data	dict[str, ChunkMetaData]	handle → 块元数据
chunkservers	dict[str, float]	server_id → 最近心跳时间戳
leases	dict[str, (str, float)]	handle → (primary, 到期时间)

4.3 ChunkServer 块存储实现

ChunkServer 以 bytearray 实现内存随机偏移写入，支持动态扩展：

Listing 3: ChunkServer.write (server/chunkserver.py)

```
1 def write(self, chunk_handle, offset, data):
2     with self.lock:
3         encoded = data.encode('utf-8')
4         if chunk_handle not in self.chunk_data:
5             self.chunk_data[chunk_handle] = bytearray()
6         buf = self.chunk_data[chunk_handle]
7         end = offset + len(encoded)
8         if len(buf) < end:
9             buf.extend(b'\x00' * (end - len(buf))) # zero-fill
10        buf[offset:end] = encoded
11        return True
```

4.4 流水线抽象 (client/stream.py)

表 3: 流水线原语类

类	职责
DataQueue	线程安全 FIFO，缓存待发送的 PacketMetaData (enqueue/dequeue/peek)
ACKQueue	追踪每个 packet 期望的副本 ACK 集合；提供 expect/ack/is_complete 等

4.5 配置参数 (gfs-site.xml)

表 4: 集群配置参数默认值

键	默认值	说明
<code>gfs.master.port</code>	9090	Master 监听端口
<code>gfs.chunk.size</code>	67108864	块大小 (64 MB)
<code>gfs.replication</code>	3	副本数
<code>gfs.lease.duration</code>	60	租约时长 (秒)
<code>gfs.heartbeat.interval</code>	5	心跳间隔 (秒)

5 测试设计与结果

5.1 测试环境

测试脚本 (`test_cluster.py`) 在进程内以守护线程形式启动完整集群，避免外部依赖：Master 端口 19090，ChunkServer 端口 19091–19093。

5.2 测试用例分组

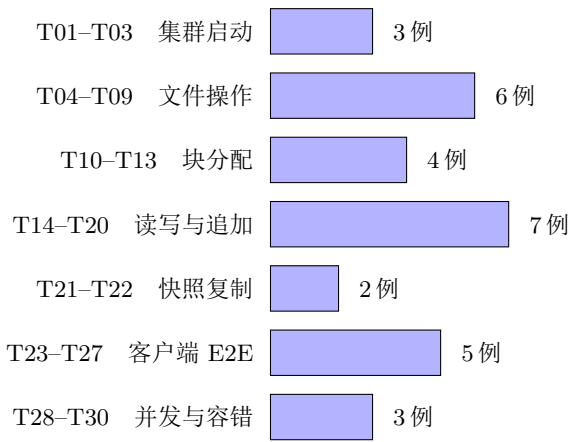


图 6: 测试用例分组分布 (共 30 例)

5.3 关键测试用例

表 5: 关键测试用例说明

用例	名称	验证点
T01	主节点连通性	Master Thrift 服务可正常连接
T03	心跳注册	heartbeat 返回 True, server_id 正确登记
T05	重复创建拒绝	同名文件二次 create 返回 False
T10	块句柄分配	返回合法 UUID 格式 chunk_handle
T12	主副本选举	primary 包含在 locations 列表内
T14	偏移写入与读回	offset=0 写入后 read 返回原始数据
T19	追加读回	append 后 read 能读到追加内容
T21	快照复制	copy_snapshot 后目标块读值等于源块
T28	并发写无数据污染	5 线程并发各写各读, 数据互不干扰

5.4 测试结果

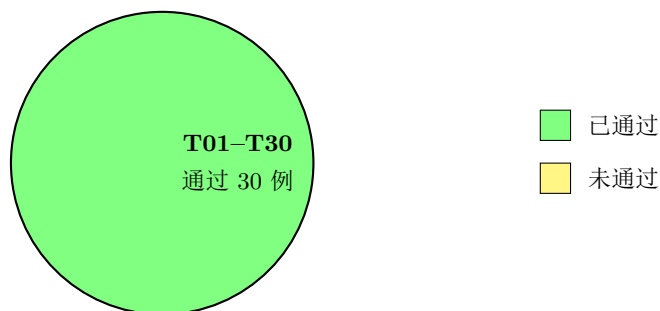


图 7: 测试通过率 (T01-T30 全部通过)

5.5 并发测试设计 (T28)

Listing 4: T28 并发数据隔离测试 (test_cluster.py)

```
1 def worker(i):
2     fname = f"/concurrent_{i}"
3     client.create(fname)
4     client.write(fname, f"data_{i}", offset=0)
5     result = client.read(fname, offset=0)
6     assert result == f"data_{i}", f"data corruption: {result}"
7
8 threads = [threading.Thread(target=worker, args=(i,)) for i in range(5)]
9 for t in threads: t.start()
10 for t in threads: t.join()
```


6 设计权衡与局限性

表 6: 设计权衡分析

特性	mGFS	GFS
存储持久化	纯内存 bytearray, 重启丢失	写前日志 + 磁盘持久化
写入流水线	客户端顺序发送至各副本	副本间链式转发
版本控制	<code>version_number</code> 固定为 0	每次写入递增, 检测过期副本
Master 容灾	单点, 无持久化	Shadow Master + 操作日志重放
快照	同节点内存 bytearray 复制	写时复制 (CoW) 跨节点快照

7 总结

mGFS 以约 600 行 Python 代码实现了 GFS 核心机制: 集中式元数据管理、UUID 分块与三副本分配、基于时间租约的主副本选举、DataQueue/ACKQueue 写入流水线抽象, 以及读操作副本故障自动转移。T01-T30 (共 30 例) 全部通过, 验证了上述机制的正确实现。

参考文献

- [1] S. Ghemawat, H. Gobioff, S.-T. Leung, *The Google File System*, SOSP 2003, pp. 29–43.
- [2] Apache Software Foundation, *Apache Thrift*, <https://thrift.apache.org/>.
- [3] thriftpy2 contributors, *thriftpy2 — Pure Python Thrift implementation*, <https://github.com/Thriftpy/thriftpy2>.